

# Analysis of the Round Trip Time of OPC UA and TSN based Peer-to-Peer Communication

Andreas Eckhardt  
Engineering Automation Systems  
Bosch Rexroth AG  
Erbach, Germany  
andreas.eckhardt@boschrexroth.de

Sebastian Müller  
Engineering Automation Systems  
Bosch Rexroth AG  
Erbach, Germany  
sebastian.mueller8@boschrexroth.de

**Abstract**—At the SPS IPC Drives in Germany a joint working group between the leading automation manufacturers announced to work together on a set of vendor independent standards for the next generation of industrial communication products. This set of standards is based on OPC UA and Time Sensitive Networking (TSN) and shall be used throughout the automation pyramid. This work contributes to analyze the performance of OPC UA and TSN. Therefore, a test framework is implemented to measure the round trip time of a peer-to-peer communication. The round trip time is an important performance indicator for controller-to-controller and controller-to-field device use cases.

**Index Terms**—OPC UA, TSN, Round trip time

## I. Introduction

The OPC Foundation launched an initiative, called OPC Field Level Communications (OPC FLC) [1]. New working groups are established to standardize the OPC UA relevant topics focused on industrial automation. The goal of OPC FLC is to standardize a next generation field level communication from the physical layer up to the application layer of the OSI reference model based on well-established open standards, such as OPC UA Publish Subscribe (PubSub) [2] as well as the Time-Sensitive Networking (TSN) standards which are developed by the IEEE 802.1 task group [3]. TSN covers layer 1 and 2 of the OSI reference model and OPC UA PubSub the ones above.

## II. Related Work and Problem Outline

OPC FLC consists of the two standards OPC UA and TSN. TSN is already topic of many scientific publications. For instance, [4] compares the performance of IEEE 802.1 TSN Time Aware Shapers and Asynchronous Traffic Shapers. [5] presents an experimental setup to evaluate the latency and jitter of time-triggered periodic frames as described in IEEE 802.1Q [6]. However, to the knowledge of the authors, there is only little research done in the field of OPC UA in combination with TSN. [7] shows an OPC UA client/server-oriented approach to achieve Round trip times (RTT) around 2 ms. The RTT is the

This work was supported by the German Federal Ministry of Education and Research (BMBF) as part of the project Future Industrial Network Architecture (FIND).

elapsed time to transit a signal over a closed circuit [8]. A RTT of 2 ms is not sufficient for many industrial automation use cases [9], [10]. [11] compares the minimum cycle times of OPC UA PubSub in combination with TSN and traditional Ethernet-based fieldbus protocols e.g. Sercos III, EtherCAT or Profinet IRT. The authors vary the payload size and the number of nodes.

## III. Contribution

Therefore, the goal of this paper is to present an implementation of a test framework which uses the TSN standards IEEE 802.1AS and IEEE 802.1Qbv and the new OPC UA specification Part 14 PubSub [2], [6], [12]. This test framework is used to measure the RTT of a peer-to-peer connection between two hardware evaluation boards communicating over TSN and OPC UA PubSub. Furthermore, several timestamps are collected in the test application to measure the processing times in the boards. This way the processing times of e.g. the encoding and decoding as well as the delivery times between the boards can be measured. Based on this knowledge the deterministic behavior of the application can be improved.

This paper is divided into eight chapters. Chapter IV gives a brief overview of OPC UA communication technologies followed by a short summary of TSN standards in chapter V. Chapter VI introduces the system design and explains the measurement approach. In chapter VII the results are given. The paper concludes in the last chapter VIII and gives an outlook on future work.

## IV. OPC UA

### A. Client/Server-oriented Communication

The OPC UA client/server concept is already well established in today's industrial automation. The client/server architecture is mostly used to exchange data from control systems to Human Machine Interface (HMI) devices of the automation. The client establishes a connection to the server and uses OPC UA services like browse, read and write to get access to the data which is provided by the server. Especially for embedded devices which are limited on resources, it can be very challenging to maintain those connections in an one-to-many scenario. Therefore, a new

standard called OPC UA PubSub was released in February 2018.

## B. OPC UA Publish Subscribe

It is based on a connectionless communication architecture which means that the publisher doesn't establish connections to the subscribers. Therefore, devices don't need to maintain those connections. In the OPC UA PubSub specification two different concepts are introduced.

1) Broker Concept: The broker is a Message Oriented Middleware that distributes messages from publishers to subscribers. In the OPC UA PubSub specification there is no Message Oriented Middleware defined but it describes the means to connect to already elsewhere defined brokers e.g. an Advanced Message Queuing Protocol broker (AMQP) or a Message Queue Telemetry Transport (MQTT) broker. If the publisher wants to publish a message, it sends the message to the topics of the broker. A topic is a broker specific term and is used to organize and filter messages. If the receiver, called subscriber, is interested in this message, it can register its interest on the corresponding topic at the broker. As soon as new messages arrive, it is forwarded to the registered subscriber. One characteristic of the broker concept is, that it can decouple the communication in space, time and synchronization. If the publisher and subscriber are decoupled in space, it means that they have no connection established between them, so they don't address each other directly. If they are decoupled in time, the publisher and subscriber don't have to be online at the same time. The broker stores the data and forwards it as soon as the subscriber is online. When they are decoupled in synchronization, the subscriber can work on a concurrent task since it gets asynchronously notified by the broker that new data is available [13].

2) Brokerless Concept: In this concept no additional middleware other than the network infrastructure is used to distribute the messages. The specification describes currently mappings for two transport protocols, which are the User Datagram Protocol (UDP) and a layer 2 communication. The protocols can use uni-/multi-/broadcast to send the messages. This concept decouples the publisher and subscriber in space and synchronization. Since there is no additional entity between the publisher and subscriber, the concept is considered to have a better performance as the broker concept.

3) Message Mappings: They define the structure of the payload in the protocol mappings. There are currently two message mappings defined in the OPC UA PubSub specification. The UA Datagram Protocol (UADP) and the JavaScript Object Notation (JSON). Both are described in the following two paragraphs.

a) UADP: It's especially designed as an optimized message protocol with the goal to bring OPC UA to the controller-to-controller and controller-to-field device level. To achieve this, it uses optimized UA binary encoding

to fulfill the requirements of those use cases. In UADP a message is called NetworkMessage and its structure is defined in the specification. The NetworkMessage can be divided in a header part and a payload part. If security is enabled, the payload is encrypted and the whole NetworkMessage is signed. A NetworkMessage is highly customizable. The user can enable or disable different features in the protocol e.g. sequence numbers, timestamps and different types of encodings or security.

b) JSON: It uses human readable text and is often used in web and enterprise software. This paper focuses on the UADP message mapping so no further description of JSON is given. Further information can be found in the OPC UA PubSub specification [2].

4) Transport Protocol Mappings: In the PubSub specification four different transport protocol mappings can be found. They are described in the following paragraphs. The focus of this paper is on the OPC UA Ethernet which is based on the Ethernet layer 2 protocol which is later used in the test framework.

a) AMQP and MQTT: They are open standard application layer protocols for Message Oriented Middlewares. Both use publish and subscribe mechanisms to deliver their messages.

b) OPC UA UDP: It uses the User Datagram Protocol (UDP) and as payload the UADP message mapping. Since UDP uses decoupled communication there is no handshake between two entities. Also UDP doesn't have ordering and error-checking of received messages as TCP offers it. Therefore, the application has to do the error handling if this is required by the use case.

c) OPC UA Ethernet: This protocol uses only the standard data-link layer information to deliver messages. It has its own ethertype 0xB62C.

## V. Time-Sensitive Networking

TSN is a set of standards developed by the TSN Task Group which is part of the IEEE 802.1 Task Group [3]. For the upcoming measurement we first want to give a brief introduction to the TSN standards 802.1Qbv and 802.1AS.

### A. IEEE 802.1Qbv

This standard is already included in IEEE 802.1Q-2018 [6] and introduces mechanisms to provide dedicated time windows for time-critical traffic. It requires clock synchronization, for instance done with the standard IEEE 802.1AS.

### B. IEEE 802.1AS with generalized Precision Time Protocol

This standard defines the synchronization of clocks [12]. There are open source implementations like PTP4I or gPTP to synchronize the clocks in the test framework [14], [15]. In this paper PTP4I is used.

## VI. System Design

### A. System Specification

The test framework is implemented on two Xilinx Zynq UltraScale+ MPSoC evaluation boards. It combines an ARM Cortex-A53 64-bit application processor unit (APU) with an ARM Cortex-R5 real-time processor unit (RPU). The RPU has two and the APU four cores [16]. By default the board has one PHY, which is connected to the APU. As addition to the default PHY, an Ethernet FMC with four 10/100/1000Mbps Ethernet ports is used [17]. Those ports are managed by the RPU. The Xilinx UltraScale+ MPSoC evaluation boards use the custom Linux version 4.14.0-xilinx-v2018.2 from Xilinx as operating system. It provides TSN capabilities which are used in the test framework.

### B. Measurement Approach

Fig. 1 gives an overview of the measurement approach. Each Xilinx board has a publisher and a subscriber. The publisher 'A' sends his UADP NetworkMessage to the subscriber 'B', then the subscriber 'B' uses a shared buffer to share the content of the NetworkMessage with the publisher 'B'. The publisher 'B' publishes a new NetworkMessage which is received by the subscriber 'A'. The delta time between  $T_{Start}$  and  $T_{End}$  equals the RTT. All the timestamps are measured in software by using the `clock_gettime()` function [18]. A more detailed description

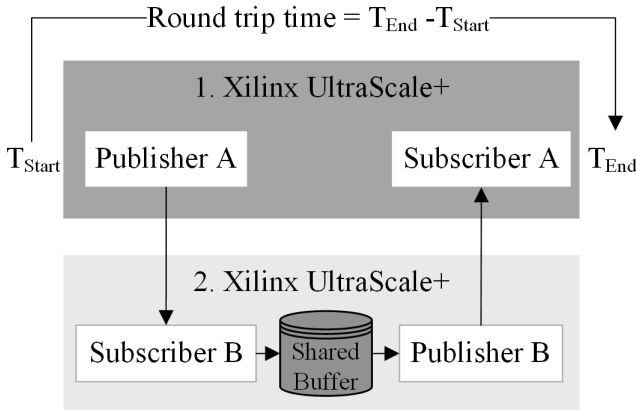


Fig. 1. Overview of the test framework concept.

of the measurement approach is displayed in Fig. 2. In the initialization phase the publishers and subscribers are getting configured. Additionally, both publishers encode the NetworkMessages and store it. After the initialization phase, the cyclic execution starts. Fig. 2 shows when and where the timestamps are taken. The `Update()`-function updates the dynamic parts of the NetworkMessage, such as the sequence number and the timestamps. The `Send()`-function copies the frame from the user space in the scheduled traffic queue in the kernel space. The `Receive()`-function copies the received frame from the kernel space to the user space. The `Decode()`-function decodes

the NetworkMessage. The `Buffer()`-functions exchange the data between the publisher and subscriber applications. For the shared buffer implementation the c++ boost library is used [19]. All the timestamps are being stored in an array. As soon as the subscriber received 10000 frames, it writes the collected timestamps in a file to be further evaluated.

### C. Test Cases

In Tab. I all the different test cases are displayed. The time between two test cycles is 1 ms, as this is a typical value for C2C use cases [9]. Thus, for test case No.1 a NetworkMessage frame size of 174 bytes is sent every 1 ms. The NetworkMessage consists of 15 DataSet Fields of type UINT64. This leads to 120 bytes user data for the first test case.

TABLE I  
Overview of the measurement cases

| No. | Frame Size | No. of Fields | Field type | User Data |
|-----|------------|---------------|------------|-----------|
| 1   | 174        | 15            | UINT64     | 120       |
| 2   | 489        | 50            | UINT64     | 400       |
| 3   | 939        | 100           | UINT64     | 800       |
| 4   | 1389       | 150           | UINT64     | 1200      |
| 5   | 1506       | 163           | UINT64     | 1304      |

### D. UADP Configuration

Fig. 3 shows the used UADP NetworkMessage. The NetworkMessage has a protocol header overhead of 21 bytes. Since the Variant type encoding is used, an additional one byte is needed to identify the type of each DataSetField in the NetworkMessage. This leads to an additional byte overhead for each DataSetField in the NetworkMessage. For instance, to send 1 byte of user data the NetworkMessage is 23 bytes long.

### E. Gate Configuration

In this test framework the cycle time of the Qbv-implementation is set to 100  $\mu$ s. Every cycle, the gate opens for the first 50  $\mu$ s to let the scheduled traffic (ST) pass. After that, the gate opens for best effort traffic until the cycle time is over. With a frame size of 1500 bytes and a bit rate of 1Gbit/s the gate needs to be open for at least 12  $\mu$ s. However, since the publisher application experiences operating system related jitter, the duration is extended to 50  $\mu$ s.

### F. PTP Synchronization

The first board is configured to be the PTP master which represents the global time and the second board as PTP slave. PTP4l is used to synchronize the slave clock with the master clock. PTP4l also shows information about the accuracy of the synchronization. Before the RTT measurement starts, the synchronization between the PTP master and the slave has to be accurate. The following

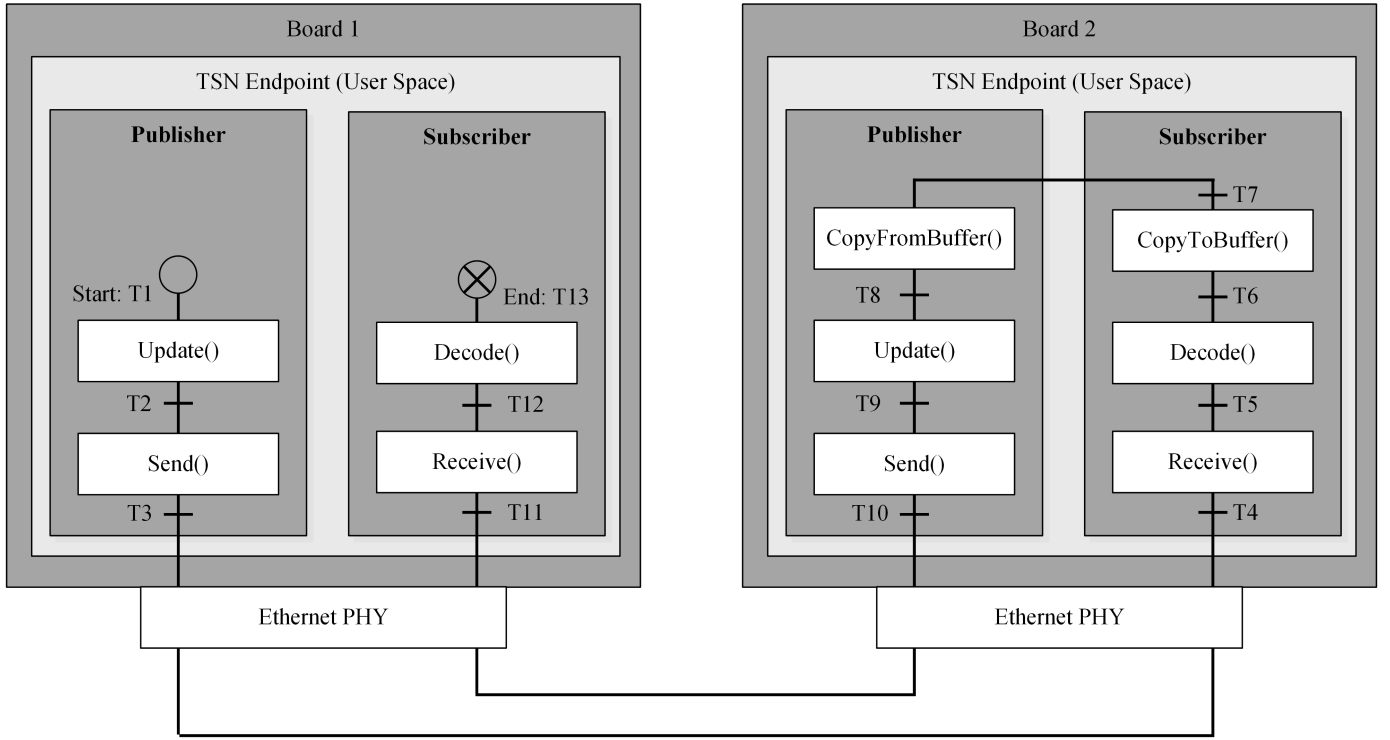


Fig. 2. Overview of the measurement approach with all the relevant timestamps.

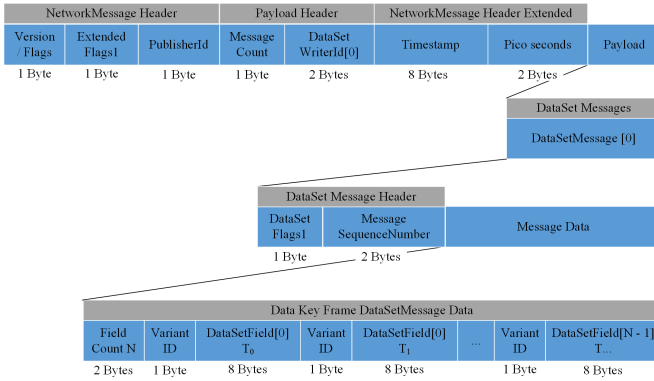


Fig. 3. An overview of the used UADP NetworkMessage.

numbers are very stable for all the test cases and are used in the test cases as reference:

- clock offset root mean square = 7 ns.
- maximum absolute clock offset = 15 ns.
- frequency adjustment of the clock in parts per billion (ppb) = 1315 ppb.
- estimated path delay value = 400 ns.

## VII. Results

### A. Round Trip Time

These tests show, among other things, the average, minimum and maximum RTT for the test cases. In Tab. II the results of the RTTs are summarized. The ratio is the sum of the average delivery times between the publishers

and subscribers, divided by the average RTT. Since the ratio is around 50% optimizing both, the delivery times and internal processing times, is equally important.

TABLE II  
Results of the round trip time measurement

| Frame size<br>[Bytes] | Round trip time |           |           | Ratio<br>Delivery/AVG |
|-----------------------|-----------------|-----------|-----------|-----------------------|
|                       | AVG [ns]        | Min. [ns] | Max. [ns] |                       |
| 174                   | 146812          | 96200     | 514440    | 0.56                  |
| 489                   | 160562          | 118000    | 523712    | 0.54                  |
| 939                   | 184509          | 148440    | 476280    | 0.48                  |
| 1389                  | 213416          | 180648    | 548040    | 0.45                  |
| 1506                  | 232320          | 187520    | 573392    | 0.45                  |

The line graph in Fig. 4 shows the RTT from 174 bytes to 1506 bytes package size. The horizontal axis represents the number of bytes of the NetworkMessage and the vertical axis represents the RTT in nanoseconds. The graph shows an increasing trend which seems to be linear between 174 bytes and 1389 bytes. After that the RTT increases on a faster rate.

The histogram in Fig. 5 represents the measured distribution of the RTT in the test case with 1506 bytes. If an application needs 100% package delivery, the cycle time can not be smaller than 580  $\mu$ s. However if an application can tolerate 0,15% of delayed packages, the cycle time could be set to 330  $\mu$ s. The jitter in the process times and context switches is responsible of the variability of the RTTs.

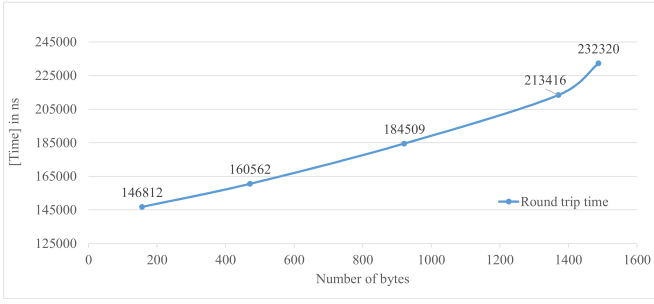


Fig. 4. RTT vs. the package size.

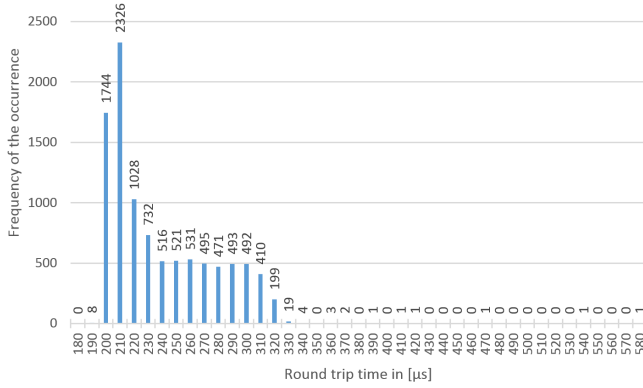


Fig. 5. Histogram of the RTT of use case No. 5 with 1506 bytes message size.

Fig. 6 shows the RTT as well as the processing times plotted against the frame sizes.

#### 1) Processing Times:

a) Encoding: After the publisher is configured, the NetworkMessage gets encoded. The measured times in Fig. 7 are the average times it takes to encode the whole NetworkMessage. In the first test case the whole frame size is 174 bytes long with 120 bytes of user data. It takes 9011 ns to encode it. The other test cases show a linear increase of the time it takes to encode increasing number of bytes to encode. In the last test case, there are 163 DataSetFields of type UINT64 which is 1304 bytes of user data.

b) Decoding: The measured times in Fig. 7 are the average times it takes to decode the whole NetworkMessage. For the first test case with a frame size of 174 bytes and 120 bytes user data, it takes 8893 ns to decode it. The decoder trend is similar to the encoder trend, though the rate of the trend is higher than the rate of the encoding. It takes approximately the same time to encode and decode the NetworkMessage. However, the next test case with 471 bytes needs 19 % longer to decode the NetworkMessage than to encode it. This trend continues with 37 % for 921 bytes, 48 % for 1371 bytes and 52 % for 1488 bytes.

### VIII. Conclusion

In conclusion this paper has given an overview of OPC UA communication architectures as well as TSN tech-

nologies. The focus has been on OPC UA PubSub, IEEE 802.1Qbv and IEEE 802.1AS. Based on those technologies a test framework has been designed and implemented which is used for testing the previously described standards on the evaluation board Zynq UltraScale+ MPSoC from Xilinx. Publishers and subscribers were implemented on the target hardware which is supporting TSN features.

The aim of the measurement is to analyze the correlation between the NetworkMessage size and the process and delivery times in the publishers and subscribers as well as to measure the RTT. The measurement shows:

- The average RTT from 146  $\mu$ s (174 bytes) to 232  $\mu$ s (1506 bytes).
- A worst-case RTT of 573  $\mu$ s (1506 bytes).
- Delivery times from user space of board 1 to user space of board 2 and vice versa make up around 50 % of the overall RTT.
- A linear correlation between the encoder/decoder time and the NetworkMessage size.
- The decoding takes longer than the encoding.

For typical control-to-control use cases with a cycle time of 1 ms and 1500 bytes the measured RTT would be already sufficient. The worst-case RTT of 1500 bytes is around 570  $\mu$ s. For use cases like control-to-field device with high performance motion applications typical requirements would be a cycle time of 125  $\mu$ s with a package size of 50 - 80 bytes. These requirements are not met yet, but with the knowledge of the internal processing and delivery times, the system design can be further optimized.

In the future, the measurement approach shall be improved by additional hardware timestamps in the kernel space. This will provide more insight of the context switches between the user space and the kernel space. The next step is to analyze the impact of certain code optimizations e.g. the impact of using different Linux schedulers and compiler optimizations.

Another step would be to apply a PREEMPT-RT patch to the Linux kernel which improves the latency of thread execution. Using an isolated CPU for the application might further improve the latency of the thread execution.

There are also optimizations that can be done in the logic of the application itself. In motion applications the whole communication cycle is time sliced to achieve the highest degree of determinism. There is a time slot for the preparation of the process data, a time slot to forward the data to the network stack and a time slot to transmit the data. Based on the presented results those time slots can now be defined and the RTT further improved. This is especially important for time-sensitive applications.

To simulate industrial networks, the number of nodes could be extended by additional publishers and subscribers to see how they impact the RTT. Furthermore, TSN-switches could be added to the network infrastructure to measure how much delay they add to the RTT.

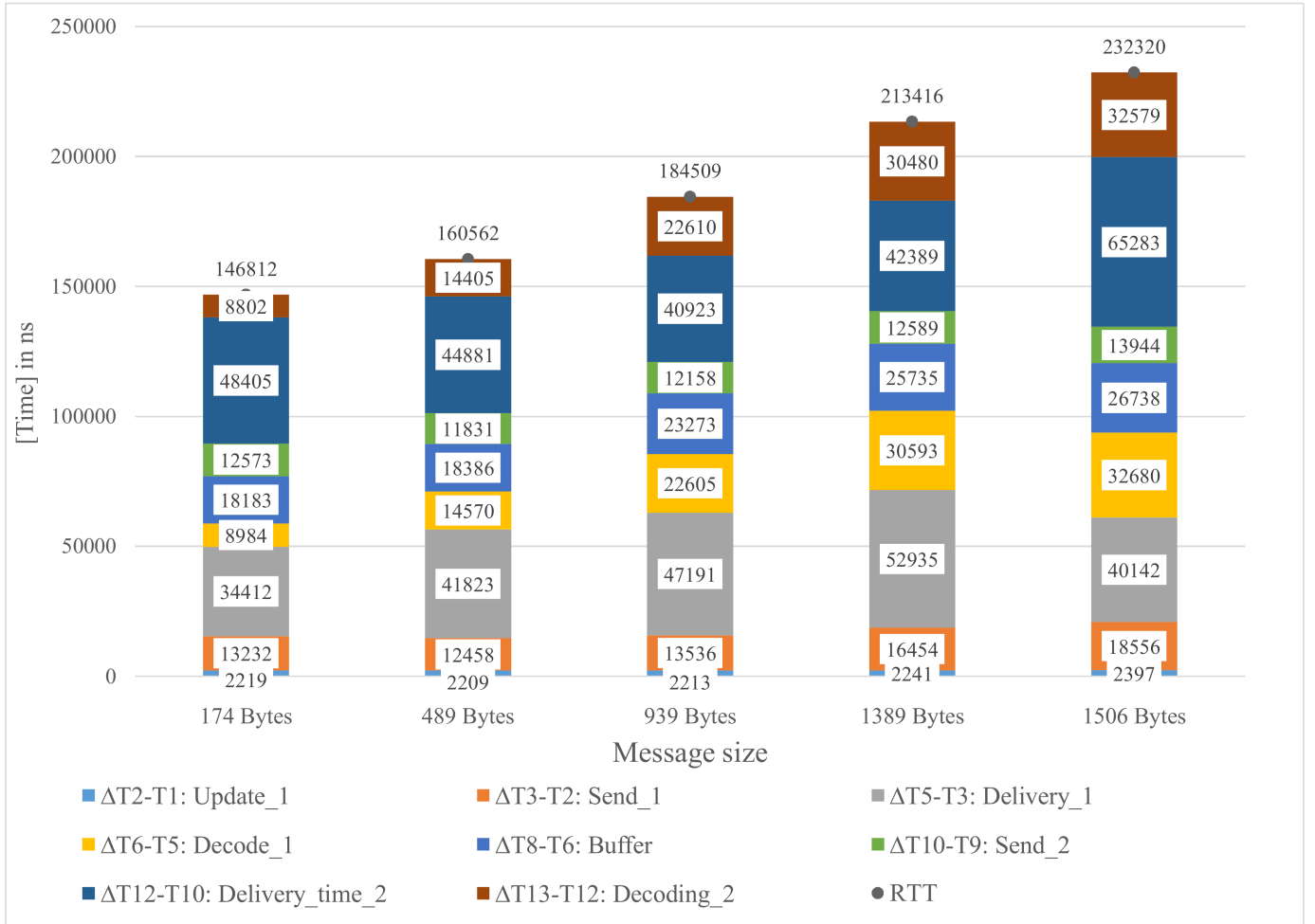


Fig. 6. RTT divided in internal processing times and delivery times.

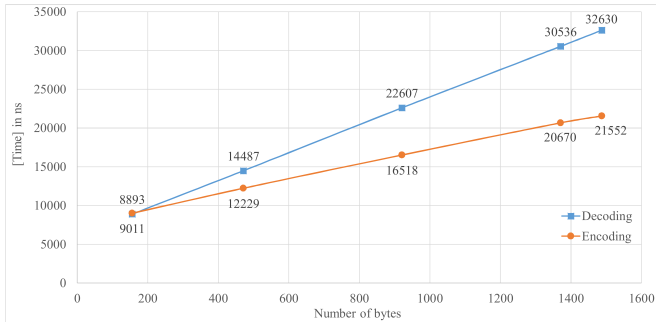


Fig. 7. Comparison between the decoding and encoding.

## References

- [1] OPC Foundation, 2019. [Online]. Available: <https://opcfoundation.org/wp-content/uploads/2018/11/OPCF-FLC-v2.pdf>
- [2] —, “OPC UA Part 14 - PubSub Specification.” [Online]. Available: <https://opcfoundation.org/developer-tools/specifications-unified-architecture/part-14-pubsub>
- [3] IEEE, “TSN Task Group,” 2019. [Online]. Available: <https://1.ieee802.org/tsn/>
- [4] A. Nasrallah, A. S. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, and H. Elbakoury, “Performance Comparison of IEEE 802.1 TSN Time Aware Shaper (TAS) and Asynchronous Traffic Shaper (ATS),” *IEEE Access*, vol. 7, pp. 44 165–44 181, 2019.
- [5] M. H. Farzaneh and A. Knoll, “Time-sensitive networking (TSN): An experimental setup,” in *2017 IEEE Vehicular Networking Conference (VNC)*. IEEE, 2017, pp. 23–26.
- [6] IEEE, “802.1Q-2018.” [Online]. Available: [https://standards.ieee.org/standard/802\\_1Q-2018.html](https://standards.ieee.org/standard/802_1Q-2018.html)
- [7] A. Gogolev, F. Mendoza, and R. Braun, “TSN-Enabled OPC UA in Field Devices,” in *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2018, pp. 297–303.
- [8] “Definition: Round Trip Time.” [Online]. Available: [https://www.its.bldrdoc.gov/fs-1037/dir-031/%5C\\_4641.htm](https://www.its.bldrdoc.gov/fs-1037/dir-031/%5C_4641.htm)
- [9] A. Eckhardt, S. Muller, and L. Leurs, “An Evaluation of the Applicability of OPC UA Publish Subscribe on Factory Automation use Cases,” in *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2018, pp. 1071–1074.
- [10] Industrial Internet Consortium, “IIC TSN Testbed Traffic Whitepaper,” 2018. [Online]. Available: [https://www.iiconsortium.org/pdf/IIC\\_TSN\\_Testbed\\_Traffic\\_Whitepaper\\_20180418.pdf](https://www.iiconsortium.org/pdf/IIC_TSN_Testbed_Traffic_Whitepaper_20180418.pdf)
- [11] D. Bruckner, R. Blair, M-P. Stanica, A. Ademaj, W. Skeffington, D. Kutscher, S. Schriegel, R. Wilmes, K. Wachswender, L. Leurs, M. Seewald, R. Hummen,

- E-C. Liu, and S. Ravikumar, "OPC UA TSN," 2019. [Online]. Available: [https://www.automationworld.com/sites/default/files/opc\\_ua\\_tsn\\_whitepaper\\_1.pdf](https://www.automationworld.com/sites/default/files/opc_ua_tsn_whitepaper_1.pdf)
- [12] IEEE, "802.1AS." [Online]. Available: [https://standards.ieee.org/standard/802\\_1AS-2011.html](https://standards.ieee.org/standard/802_1AS-2011.html)
  - [13] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Computing Surveys*, vol. 35, no. 2, pp. 114–131, 2003.
  - [14] Linux man page, "ptp4l." [Online]. Available: <https://linux.die.net/man/8/ptp4l>
  - [15] AVnu, "gPTP." [Online]. Available: <https://github.com/AVnu/OpenAvnu>
  - [16] Xilinx, "UltraScale+ MPSoC." [Online]. Available: [https://www.xilinx.com/support/documentation/data\\_sheets/ds890-ultrascale-overview.pdf](https://www.xilinx.com/support/documentation/data_sheets/ds890-ultrascale-overview.pdf)
  - [17] Opsero Electronic Design Inc., "Ethernet FMC." [Online]. Available: <http://ethernetfmc.com/specifications>
  - [18] Linux man page, "Timestamping." [Online]. Available: [https://linux.die.net/man/3/clock\\_gettime](https://linux.die.net/man/3/clock_gettime)
  - [19] "Boost C++ library." [Online]. Available: <https://www.boost.org/>